UC SAN DIEGO / ECE 251C

14

Finite wavelet transforms and signal recovery

Executive summary

Implementing the DWT on a finite signal requires a boundary convention. We compare linear and circular convolution, count the retained coefficients, and derive the arithmetic cost of the periodic transform. The same construction extends to images by filtering along rows and columns. We then use sparsity to motivate nonlinear processing of wavelet coefficients and place denoising in the more general setting of signal recovery. Least squares, regularization, and the matrix representation of the finite DWT provide the connection to linear algebra and optimization.

14.1 Computational and numerical considerations

The wavelet construction has so far been expressed using sequences indexed by all integers. In a numerical implementation, we store only a finite record. Recall that the DWT is obtained by iterating the low-pass branch of a perfect-reconstruction analysis bank: each stage produces a coarser approximation and the details needed to recover the finer one. We retain the details and pass only the approximation coefficients to the next stage. We must now specify how the filters act when they reach either end of the stored record.

At resolution $i+1$, the analysis equations are

$$a_i[n] = (h_0 * a_{i+1})[2n], \qquad d_i[n] = (h_1 * a_{i+1})[2n]. \tag{14.1}$$

For an N-point input, it is tempting to conclude that downsampling gives $N/2$ coefficients in each branch. This overlooks the filtering step: linear convolution can increase the length before decimation. The output count therefore depends on the boundary convention and on which convolution outputs are retained. We begin by making these choices explicit.

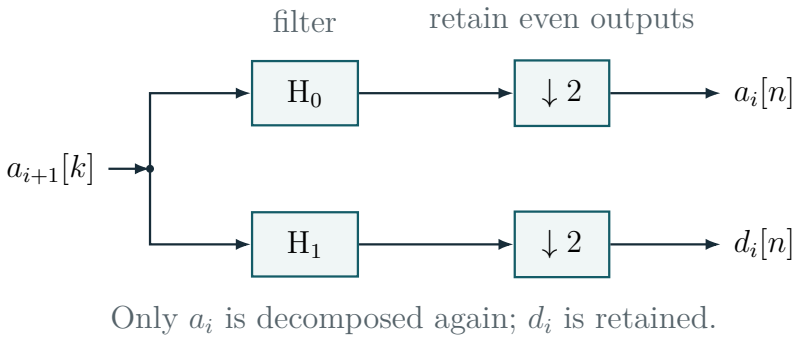


Figure 14.1: One stage of the DWT. Filtering produces the approximation and detail branches, and downsampling retains their even-indexed outputs. Only the approximation branch is decomposed again.

14.2 Length of a convolution

An N-point signal has N stored samples, some of which may be zero. Suppose $x[n]$ is supported in $\{0, \dots, N-1\}$ and $h[n]$ in $\{0, \dots, L-1\}$. Extend both sequences by zero. Their convolution is

$$y[k] = (h * x)[k] = \sum_{r=0}^{L-1} h[r]x[k-r].$$

A nonzero summand requires both $0 \le r \le L-1$ and $0 \le k-r \le N-1$. Therefore

$$0 \le k \le N + L - 2.$$

Thus the convolution has length at most $N + L - 1$. When the first and last entries of both sequences are nonzero,

$$y[0] = h[0]x[0] \neq 0, \qquad y[N+L-2] = h[L-1]x[N-1] \neq 0,$$

and the length is exactly $N + L - 1$. Some entries between these endpoints may be zero.

Keeping even indices $k = 2n$ from that full array gives

$$0 \le n \le \left\lfloor \frac{N+L-2}{2} \right\rfloor.$$

Hence the number of retained positions *per branch* is

$$\boxed{\left\lfloor \frac{N+L-2}{2} \right\rfloor + 1 = \left\lceil \frac{N+L-1}{2} \right\rceil.} \tag{14.2}$$

For even N and even L, this becomes $(N + L)/2$. Computing both full branches then produces $N + L$ positions, rather than N.

The two branches can therefore contain more stored coefficients than the input has samples. These coefficients cannot all be independent, since they are obtained from the same N numbers. However, a count of the redundancy does not identify which outputs may be discarded without losing information. The Haar example makes this distinction explicit.

14.3 Example: a finite-length Haar transform

For this boundary calculation, use the *causal* analysis filters

$$h_0^{\mathrm{c}}[k] = \frac{\delta[k] + \delta[k-1]}{\sqrt{2}}, \qquad h_1^{\mathrm{c}}[k] = \frac{\delta[k] - \delta[k-1]}{\sqrt{2}}.$$

Consider the six-point signal $x[0] = 1$, $x[1] = 3$, $x[2] = x[3] = 2$, and $x[4] = x[5] = 4$, extended by zero. Each convolution has seven entries. Downsampling the full output at even indices gives four approximation coefficients and four detail coefficients.

Which coefficients are redundant? Write the six input entries as $x_0, \ldots, x_5$. With causal filters and even-output retention,

$$a^{\mathrm{c}}[n] = \frac{x[2n] + x[2n-1]}{\sqrt{2}}, \qquad d^{\mathrm{c}}[n] = \frac{x[2n] - x[2n-1]}{\sqrt{2}}.$$

The four output pairs are

n	$\sqrt{2}\, a^{\mathrm{c}}[n]$	$\sqrt{2}\, d^{\mathrm{c}}[n]$
0	x_0	x_0
1	$x_2 + x_1$	$x_2 - x_1$
2	$x_4 + x_3$	$x_4 - x_3$
3	x_5	$-x_5$

Thus $d^{\mathrm{c}}[0] = a^{\mathrm{c}}[0]$ and $d^{\mathrm{c}}[3] = -a^{\mathrm{c}}[3]$. One may retain all four approximation coefficients and only the two interior detail coefficients. They recover the input by

$$x_0 = \sqrt{2}\, a^{\mathrm{c}}[0], \qquad x_5 = \sqrt{2}\, a^{\mathrm{c}}[3],$$

$$x_{2n} = \frac{a^{\mathrm{c}}[n] + d^{\mathrm{c}}[n]}{\sqrt{2}}, \qquad x_{2n-1} = \frac{a^{\mathrm{c}}[n] - d^{\mathrm{c}}[n]}{\sqrt{2}}, \quad n = 1, 2.$$

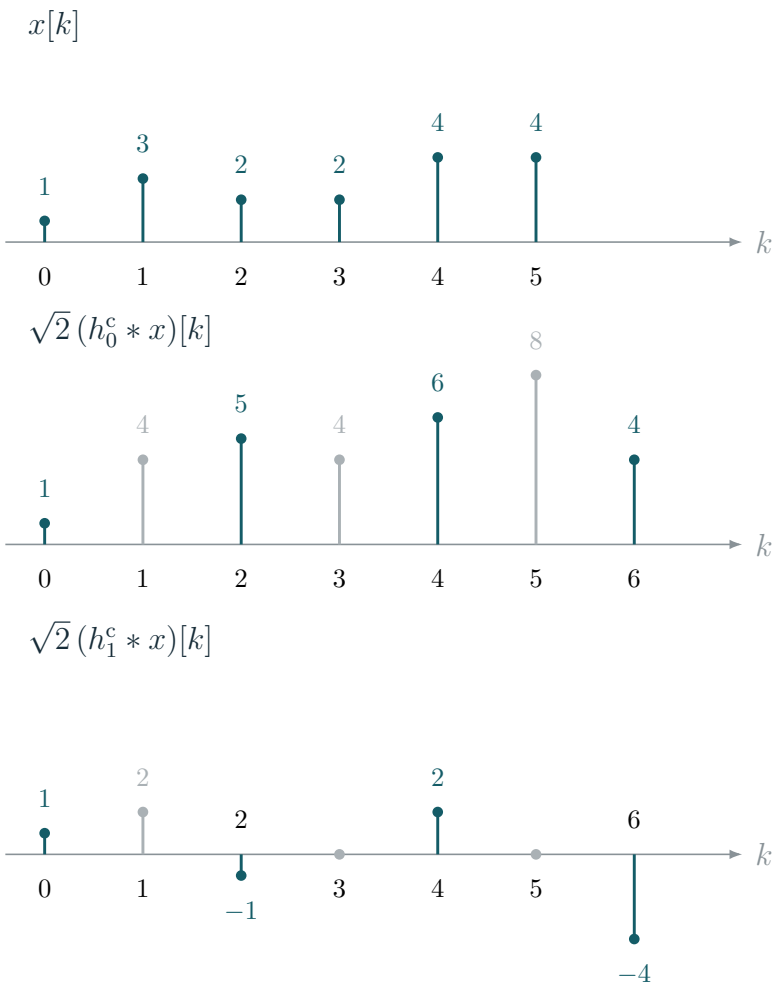


Figure 14.2: A six-sample example with causal Haar filters. Filtered values are shown after multiplication by $\sqrt{2}$. In the filtered sequences, teal marks the retained even outputs and gray marks the discarded odd outputs. The last even outputs contain the last input sample.

Thus six coefficients suffice to recover the six input samples. The particular coefficients retained depend on the alignment: in this causal convention, both final outputs contain x_5, and discarding both would lose that sample.

The adjacent-pair convention used earlier. The zero-delay analysis filters $h_\ell[k] = g_\ell[-k]$ for $g_0 = (1,1)/\sqrt{2}$ and $g_1 = (1,-1)/\sqrt{2}$ give

$$a[n] = \frac{x[2n] + x[2n+1]}{\sqrt{2}}, \qquad d[n] = \frac{x[2n] - x[2n+1]}{\sqrt{2}}. \tag{14.3}$$

Here the samples are grouped into the pairs $(x[0], x[1])$, $(x[2], x[3])$, and $(x[4], x[5])$. Consequently, $n = 0, 1, 2$ gives three approximation coefficients and three detail coefficients. This is the zero-delay convention from the earlier lectures, which we use below.

14.4 The periodic DWT

Longer filters produce more boundary coefficients under linear convolution. A periodic implementation avoids this increase by interpreting the finite record as one period of a sequence and replacing each convolution by circular convolution. With even input length, the two decimated branches then contain exactly as many coefficients in total as the input.

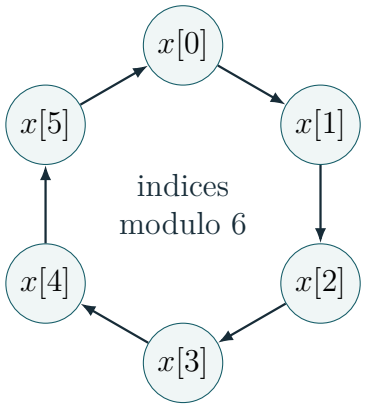
For an N-point signal, define

$$x_{\mathrm{per}}[k] = x[k \bmod N], \qquad k \in \mathbb{Z},$$

where $k \bmod N$ is the representative in $\{0, \dots, N-1\}$. For a finitely supported filter h, define

$$(h \circledast_N x)[k] = \sum_{r \in \operatorname{supp} h} h[r]x[(k-r) \bmod N], \qquad 0 \le k < N. \tag{14.4}$$

The modulo operation specifies the value of every sample needed by the filter. For example, index -1 refers to the last stored sample, and index N refers to the first. All filter taps are included in the sum, even when the filter is longer than the current coefficient sequence; several taps may then refer to the same sample in the period.



$x[-1] = x[5]$

$x[6] = x[0]$

$x[k+6] = x[k]$

The record length stays six. Wrapping changes its boundary model.

Figure 14.3: Circular convolution treats the endpoints as neighbors. Only one period is stored; sample values outside this period are obtained by wrapping their indices.

For even N, the periodic analysis stage is

$$\begin{aligned} a[n] &= (h_0 \circledast_N x)[2n], \\ d[n] &= (h_1 \circledast_N x)[2n], \end{aligned} \qquad 0 \le n < N/2. \tag{14.5}$$

There are $N/2$ coefficients in each branch, so the total remains N. This is analogous to the periodic extension implicit in the DFT. The synthesis bank must use the

same convention. A perfect-reconstruction delay then becomes a circular shift, which can be compensated to recover the original alignment.

14.5 Boundary effects

The exact coefficient count comes with a modeling choice: periodic extension treats the last and first samples as neighbors. If those values do not match, it introduces an artificial jump, just as periodic extension can introduce boundary artifacts in a DFT representation. Wavelets near this boundary respond to the jump just as they respond to a transition within the signal.

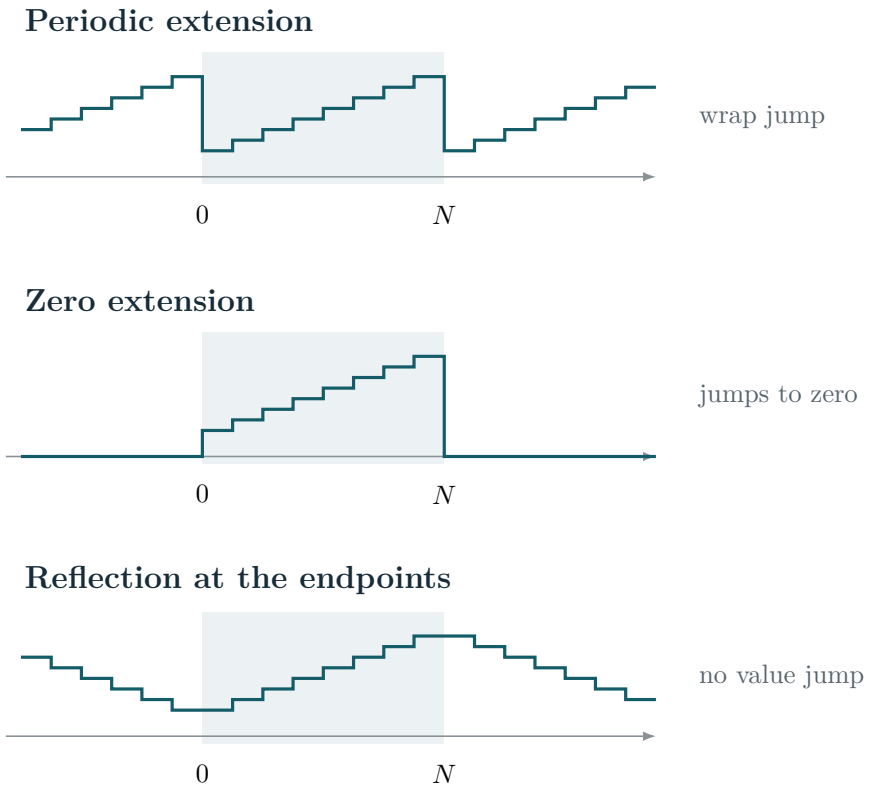


Figure 14.4: The shaded segment is the stored record. Periodization, zero extension, and reflection give different continuations beyond its endpoints. These continuations affect the wavelet coefficients whose filters overlap the boundary.

Zero padding places the signal inside a longer sequence of zeros, so the periodic wrap can occur in a zero region. It does not remove the transitions from nonzero endpoints to zero. For a single convolution of lengths N and L, padding to at least $N + L - 1$ prevents wrap-around of the full output.

Reflection provides another continuation of the record, rather than treating its endpoints as neighbors. The continuation must be used consistently with the

chosen boundary analysis and synthesis operators. For the exact coefficient counts and arithmetic costs below, we use periodic extension.

For the periodic implementation of a perfect-reconstruction bank, the original record is recovered, up to its specified circular delay, when the coefficients are left unchanged. Other extensions require their corresponding reconstruction rules and enough retained boundary coefficients. Boundary effects are particularly important after coefficient modification: thresholding may alter a coefficient caused by the chosen continuation rather than by a feature inside the record.

14.6 Computational complexity

Consider direct evaluation of a length-L FIR filter. Each output is formed from L products and $L-1$ additions, counting all tap positions and without exploiting special coefficients. Thus a circular convolution with an N-point record uses NL multiplications and $N(L-1)$ additions. This count makes the dependence on both the signal length and the filter length explicit.

For a decimated output, compute only

$$(h \circledast_N x)[2n] = \sum_{r \in \operatorname{supp} h} h[r]x[(2n-r) \bmod N], \qquad 0 \le n < N/2.$$

There are only $N/2$ retained outputs, so direct evaluation requires $NL/2$ multiplications and $N(L-1)/2$ additions. The saving comes from computing only the outputs that will be used. Computing the full convolution and then discarding the odd-indexed outputs would perform unnecessary work.

For the DWT, suppose both analysis filters have length L and $N = 2^I$. The first stage computes both a_{I-1} and d_{I-1}. Each has $N/2$ entries, so the total is LN multiplications. Continuing along the low-pass branch gives the following counts.

Input	Length	Outputs	Each length	Multiplications
a_I	N	a_{I-1}, d_{I-1}	$N/2$	LN
a_{I-1}	$N/2$	a_{I-2}, d_{I-2}	$N/4$	$LN/2$
a_{I-2}	$N/4$	a_{I-3}, d_{I-3}	$N/8$	$LN/4$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
a_{i+1}	2^{i+1}	a_i, d_i	2^i	$L2^{i+1}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
a_1	2	a_0, d_0	1	$2L$

At resolution i, both a_i and d_i have 2^i entries. Equivalently, after r stages, a_{I-r} and d_{I-r} each have $N/2^r$ entries.

14.7 Total cost of the DWT

The approximation sequence passed to the next stage has half as many entries as the current input. As a result, the arithmetic performed over all stages forms a geometric sum rather than I copies of the first-stage cost. Summing the multiplication counts gives

$$\begin{aligned} C_{\text{mult}} &= LN + \frac{LN}{2} + \frac{LN}{4} + \cdots + 2L \\ &= L \sum_{i=0}^{I-1} 2^{i+1} = 2L(2^I - 1) = \boxed{2L(N-1)}. \end{aligned} \tag{14.6}$$

Likewise,

$$C_{\text{add}} = \boxed{2(L-1)(N-1)}. \tag{14.7}$$

These counts are for direct evaluation of the two filters. Special filter coefficients may allow fewer operations. If the filter lengths are L_0 and L_1, the multiplication count is $(L_0 + L_1)(N-1)$.

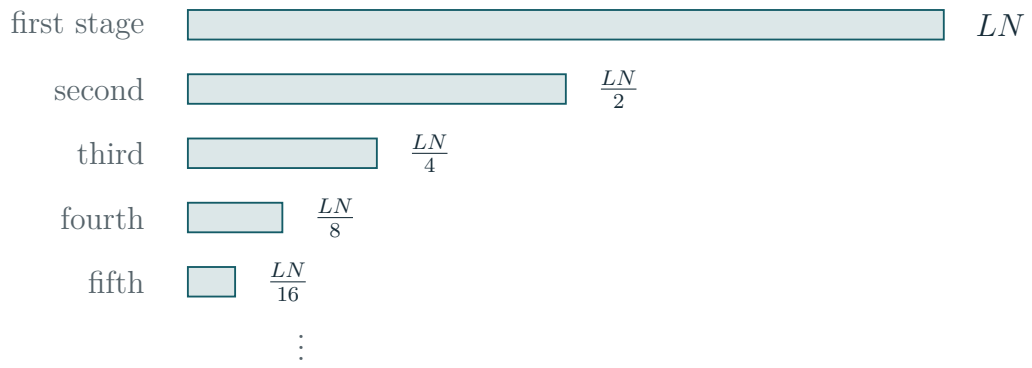


Figure 14.5: Work per stage in direct evaluation of both branches. Each stage has half as many input coefficients as the preceding one. The sum is less than twice the first-stage cost.

Thus the arithmetic cost is $O(LN)$. For a fixed pair of FIR filters, the tap length L is independent of N, and the cost is $O(N)$. The fixed-filter assumption matters: saying only that L is small relative to N would not by itself imply linear asymptotic complexity.

The standard radix-two FFT costs $O(N \log N)$. For the DWT, the work over all resolutions forms a geometric sum: the second stage costs half as much as the first, the third costs one-quarter as much, and so on. Equivalently, its cost satisfies

$$T_{\mathrm{DWT}}(N) = T_{\mathrm{DWT}}(N/2) + O(LN).$$

Thus the presence of $I = \log_2 N$ stages does not introduce a logarithmic factor. The total work is less than twice the first-stage cost.

14.8 The total number of coefficients

After the full decomposition, we keep the coarsest approximation a_0 and all the detail sequences $d_0, \ldots, d_{I-1}$. One ordering of the output coefficients is

$$(a_0[0],\ d_0[0],\ d_1[0], d_1[1],\ \ldots,\ d_{I-1}[0], \ldots, d_{I-1}[2^{I-1}-1]). \tag{14.8}$$

There are exactly

$$\underbrace{1}_{a_0} + \underbrace{\sum_{i=0}^{I-1} 2^i}_{\text{all details}} = 1 + (2^I - 1) = N$$

entries. For $N = 8$, the three stages are

$$\begin{aligned} a_3\ (8) &\longrightarrow a_2\ (4),\ d_2\ (4), \\ a_2\ (4) &\longrightarrow a_1\ (2),\ d_1\ (2), \\ a_1\ (2) &\longrightarrow a_0\ (1),\ d_0\ (1). \end{aligned}$$

The retained lengths are $1 + 1 + 2 + 4 = 8$, while the direct multiplication count is $8L + 4L + 2L = 14L$.

The DWT itself has not compressed the signal: it has replaced N samples by N coefficients. The benefit is that the energy may be concentrated in relatively few of these coefficients. Compression then becomes possible by approximating small coefficients by zero and quantizing and encoding the retained values. These later operations, rather than the invertible transform, discard information.

The intermediate approximations $a_1, \ldots, a_{I-1}$ need not be stored as additional outputs. Each is decomposed into the next approximation and detail sequence. The required storage is therefore $O(N)$.

For reconstruction, combine a_0 and d_0 to recover a_1, then combine a_1 and d_1 to recover a_2, and continue. The inverse DWT also costs $O(LN)$.

14.9 Two-dimensional DWTs

The one-dimensional algorithm extends directly to images by treating the two coordinates in succession. First transform every row, and then transform every column of the resulting arrays. This is a *separable* two-dimensional DWT: its filters are products of one-dimensional filters, and the computation is a sequence of one-dimensional operations. All convolutions in this section use periodic boundaries.

Let $X[r, c]$ be a $(2N) \times (2N)$ image, where r is the row index and c is the column index. Filtering each row and downsampling along the column index gives

$$U_\ell[r, m] = \sum_k h_\ell[k] X[r, (2m - k) \bmod 2N], \qquad \ell = 0, 1.$$

Apply the analysis bank to each column of these two arrays:

$$C_{\ell s}[n, m] = \sum_q h_s[q] U_\ell[(2n - q) \bmod 2N, m]. \tag{14.9}$$

Here $0 \le n < N$ and $0 \le m < N$. Substitution gives the separable formula

$$C_{\ell s}[n, m] = \sum_{q,k} h_s[q] h_\ell[k] X[(2n - q) \bmod 2N, (2m - k) \bmod 2N].$$

The four outputs are $\mathrm{LL} = C_{00}$, $\mathrm{LH} = C_{01}$, $\mathrm{HL} = C_{10}$, and $\mathrm{HH} = C_{11}$. The first letter indicates the filter applied along each row; the second indicates the filter applied along each column. Each output is an $N \times N$ array.

14.10 Example: the two-dimensional Haar transform

Use the adjacent-pair Haar convention in (14.3). For a pixel block

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

the horizontal low-pass values are $(a + b)/\sqrt{2}$ and $(c + d)/\sqrt{2}$, and the horizontal high-pass values are $(a - b)/\sqrt{2}$ and $(c - d)/\sqrt{2}$. A second average–difference operation yields

$$\boxed{\begin{aligned} \mathrm{LL} &= (a + b + c + d)/2, \quad \mathrm{LH} = (a + b - c - d)/2, \\ \mathrm{HL} &= (a - b + c - d)/2, \quad \mathrm{HH} = (a - b - c + d)/2. \end{aligned}} \tag{14.10}$$

Each one-dimensional stage contributes a factor $1/\sqrt{2}$, giving the factor $1/2$. The LL coefficient describes the local average, up to normalization. The other three coefficients describe differences across rows, across columns, or in both directions.

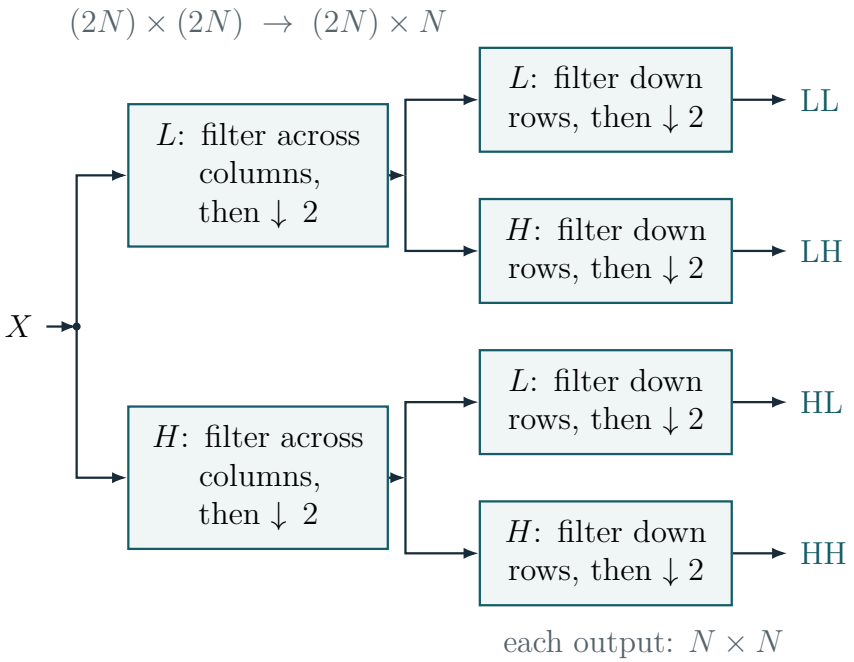


Figure 14.6: A separable two-dimensional analysis bank. The first letter specifies the horizontal filter and the second specifies the vertical filter. Each branch contains an $N \times N$ array, so the total coefficient count remains $(2N)^2$.

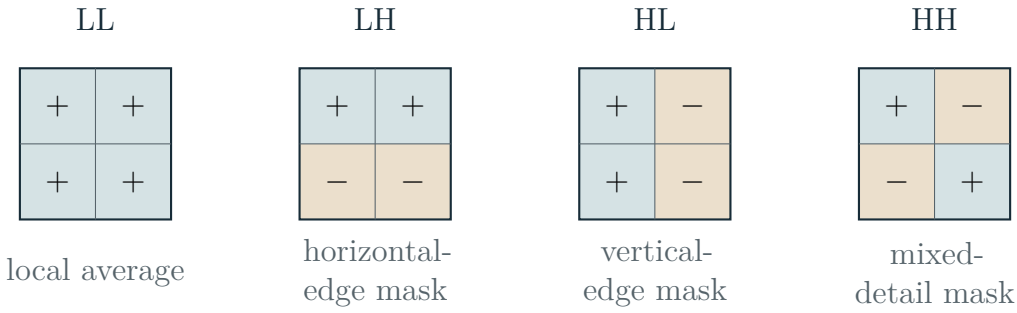


Figure 14.7: The four Haar masks on a 2×2 pixel block. Each sign represents $\pm 1/2$, and the masks are orthonormal. The detail masks distinguish changes across rows, across columns, and in both directions.

For example, $\left[\begin{smallmatrix} 1 & 3 \\ 1 & 3 \end{smallmatrix}\right]$ gives

$$(\mathrm{LL}, \mathrm{LH}, \mathrm{HL}, \mathrm{HH}) = (4, 0, -2, 0).$$

The left-to-right change appears in HL; a top-to-bottom change appears in LH. A diagonal edge can contribute to several bands.

Adding and subtracting the four equations gives the inverse:

$$a = (\mathrm{LL} + \mathrm{LH} + \mathrm{HL} + \mathrm{HH})/2, \quad b = (\mathrm{LL} + \mathrm{LH} - \mathrm{HL} - \mathrm{HH})/2,$$
$$c = (\mathrm{LL} - \mathrm{LH} + \mathrm{HL} - \mathrm{HH})/2, \quad d = (\mathrm{LL} - \mathrm{LH} - \mathrm{HL} + \mathrm{HH})/2.$$

The four masks are orthonormal, so

$$a^2 + b^2 + c^2 + d^2 = \mathrm{LL}^2 + \mathrm{LH}^2 + \mathrm{HL}^2 + \mathrm{HH}^2.$$

14.11 Multilevel image decompositions

One stage maps a $(2N) \times (2N)$ image into four $N \times N$ arrays. The total coefficient count is unchanged:

$$4N^2 = (2N)^2.$$

The LL array approximates the image at a lower resolution. Apply the analysis bank again to this array and retain the other three arrays as details. Repeating this procedure gives a multiresolution decomposition, as in the one-dimensional DWT.

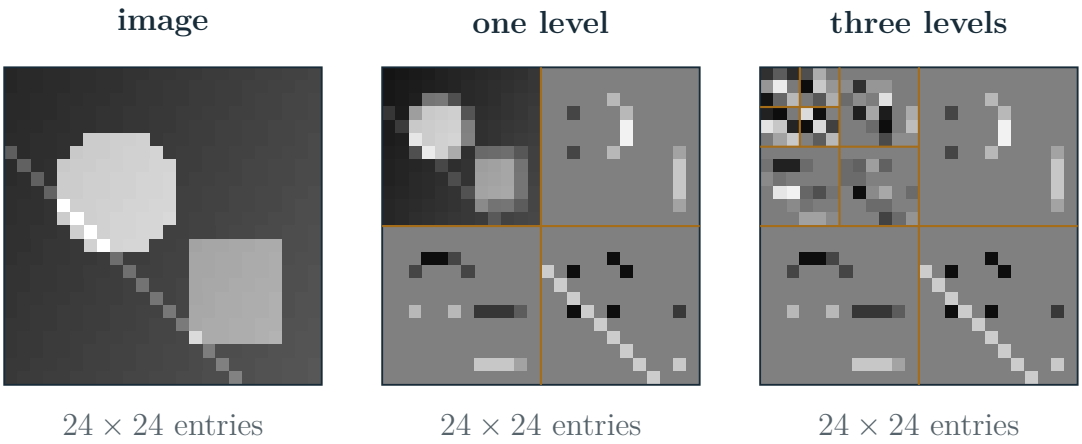


Figure 14.8: A computed Haar decomposition of a 24×24 image. The top row contains LL, HL and the bottom row LH, HH. Only the upper-left block is decomposed again. Each detail band uses its own display scale, with zero in midgray.

For a $2^I \times 2^I$ image, the final approximation has one coefficient, and resolution i has three detail arrays with 4^i entries each. Thus

$$1 + 3 \sum_{i=0}^{I-1} 4^i = 4^I.$$

For the displayed 24×24 example, three stages leave a 3×3 approximation, with $9 + 3(9 + 36 + 144) = 576$ coefficients in total.

Cost per pixel. For a $(2N) \times (2N)$ input, the horizontal passes cost $4LN^2$ multiplications and the vertical passes cost another $4LN^2$. The next LL block has one-quarter as many entries. For a square image of side $S = 2^I$, the full count is

$$2LS^2 \sum_{r=0}^{I-1} 4^{-r} = \frac{8}{3} L(S^2 - 1) = O(LS^2). \tag{14.11}$$

For fixed filters, the cost is linear in the number of pixels, not in the side length. Reconstruction reverses the vertical and horizontal synthesis steps.

14.12 Nonlinear processing of wavelet coefficients

Recall that the wavelet coefficients of a piecewise-smooth signal are often zero or small away from its transitions. This motivates a simple approach to compression and denoising: compute the DWT, modify the coefficients, and reconstruct. The representation is useful because a small number of coefficients can account for much of the signal's energy.

A transform alone cannot remove noise. Applying the DWT followed immediately by its inverse returns the input exactly, including any noise in it. Denoising requires a coefficient-processing step between analysis and synthesis. Sparsity provides a reason to consider suppressing coefficients of small magnitude, but the effect of this choice must be judged relative to both the signal and the noise.

For example, *hard thresholding* sets small coefficients to zero and leaves the remaining coefficients unchanged. For a threshold $\tau > 0$, define

$$\eta_\tau(z) = \begin{cases} 0, & |z| \le \tau, \\ z, & |z| > \tau. \end{cases}$$

This rule is nonlinear. Indeed, $\eta_\tau(3\tau/4) = 0$, whereas $\eta_\tau(3\tau/2) = 3\tau/2$. Thus applying the rule to a sum need not give the sum of the separately thresholded coefficients.

Noise perturbs the wavelet coefficients, so coefficients that vanish for the clean signal generally need not vanish for the observations. Thresholding can suppress these perturbations, but it can also discard small signal coefficients. Choosing a threshold therefore involves a tradeoff between reducing noise and retaining signal detail; sparsity alone does not make every small observed coefficient pure noise.

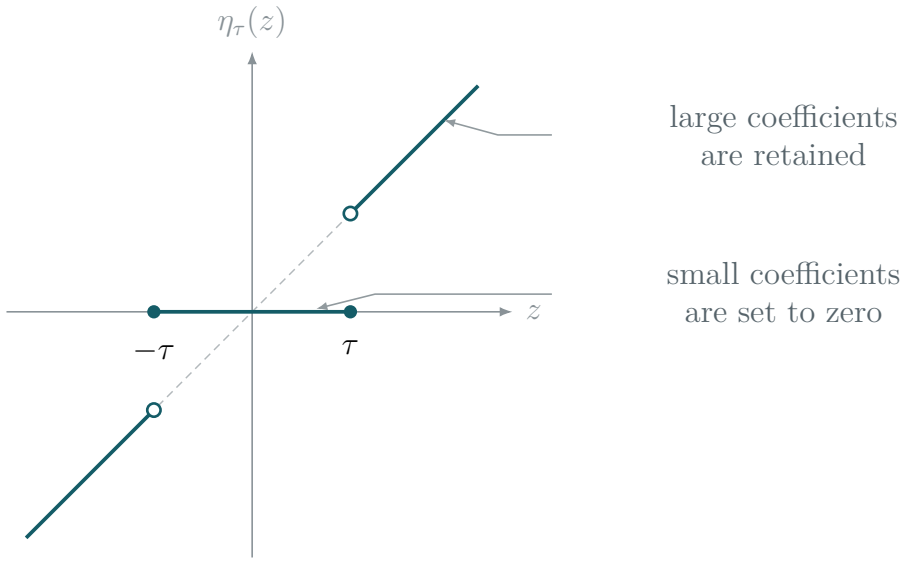


Figure 14.9: Hard thresholding with the convention $\eta_\tau(z) = 0$ at $|z| = \tau$. The dashed diagonal is the identity map. The thresholding rule is nonlinear and discontinuous; the DWT itself is linear.

Historical perspective. Wavelet shrinkage and thresholding methods, developed by Donoho and collaborators in the 1990s, made sparse representations a central tool for signal estimation. Compressed sensing, associated with the work of Candès, Romberg, Tao, and Donoho around 2006, extended this emphasis on sparsity to recovery from limited measurements. From the 2010s onward, deep learning brought a further emphasis on learning useful representations from data. These developments form part of a broader change in signal processing: alongside the classical study of z-transforms and DTFTs, one studies how signal structure can be used to recover an unknown input. Wavelets helped establish this viewpoint, which leads naturally to solving linear systems exactly or approximately, and to least squares and regularization.

14.13 Denoising, estimation, and recovery of signals

Suppose we observe

$$\boxed{y[n] = x[n] + \varepsilon[n],} \tag{14.12}$$

where $x[n]$ is the signal of interest, $\varepsilon[n]$ is additive noise, and $y[n]$ is the observed signal. Our goal is to estimate $x[n]$ from $y[n]$. This problem is called *denoising*. Wavelet thresholding is one approach: it uses the structure of the signal's wavelet coefficients to distinguish the signal from the noise.

More generally, we often observe $x[n]$ through a channel, an acquisition device, or a sampling system. Denote this system by H. If noise is added to its output, the observation model is

$$y[n] = (\mathrm{H}x)[n] + \varepsilon[n].$$

The problem is now to estimate the input of the system from its noisy output.

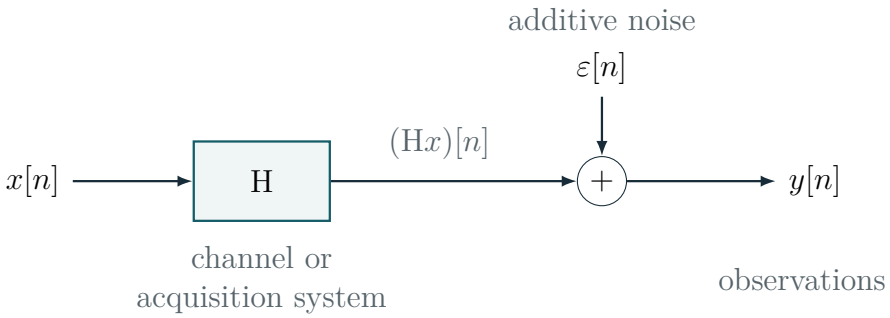


Figure 14.10: The sequence-domain observation model. The system transforms the signal, then additive noise corrupts its output. We seek to estimate the input from these observations.

The system H need not be time invariant. In many acquisition models it is nevertheless linear. This distinction is important: convolution describes an LTI model, whereas a matrix can describe any linear map between two finite-dimensional signal spaces. We now use that more general description.

We have so far studied how to represent signals and reconstruct them from their coefficients. Signal recovery introduces a further question: how should we use the observations, together with prior knowledge about the signal, to construct an estimate? This leads to systems of equations, least squares, and regularization.

14.14 From a finite sequence to a vector

For an N-point real signal, collect the samples into a column vector:

$$x = \begin{pmatrix} x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{pmatrix} \in \mathbb{R}^N.$$

The vector contains the same samples as the sequence; no new signal has been introduced. What changes is the representation of the acquisition system. A linear system that maps N input samples to M observations is represented by an $M \times N$ matrix. The observation model is therefore

$$\boxed{y = \mathrm{H}x + \varepsilon,} \tag{14.13}$$

where

$y \in \mathbb{R}^M$	observations
$\mathrm{H} \in \mathbb{R}^{M \times N}$	system matrix
$x \in \mathbb{R}^N$	signal of interest
$\varepsilon \in \mathbb{R}^M$	additive noise

In particular, each row of H specifies one measurement:

$$y[m] = \sum_{n=0}^{N-1} H_{mn} x[n] + \varepsilon[m], \qquad m = 0, \dots, M-1.$$

The noise has the same dimension as the observations, since these vectors are added. The number of measurements M need not equal the number of unknown signal samples N.

Signal recovery as an inverse problem

Given the observations y and the system matrix H, estimate the signal x.

Denoising is the special case $M = N$ and $\mathrm{H} = \mathrm{I}_N$. More generally, the matrix need not be square or invertible. The phrase "inverse problem" refers to recovering the input from the output; it does not assume that an ordinary matrix inverse exists.

14.15 Least squares and regularization

If there is no noise, the observations satisfy

$$y = \mathrm{H}x.$$

We therefore seek a signal consistent with these equations. When noise is present, exact consistency may no longer be possible. A natural approach is to choose x so that its predicted measurements $\mathrm{H}x$ are close to y. This gives the *least-squares problem*

$$\boxed{\min_{x \in \mathbb{R}^N} \|y - \mathrm{H}x\|_2^2.} \tag{14.14}$$

The vector $y - \mathrm{H}x$ is called the *residual*. Its squared Euclidean norm is

$$\|y - \mathrm{H}x\|_2^2 = \sum_{m=0}^{M-1} \big| y[m] - (\mathrm{H}x)[m] \big|^2.$$

Thus least squares measures the total squared discrepancy between the observations and the measurements predicted by a candidate signal. The data y and the matrix H are fixed; the unknown vector x is the optimization variable. No assumption that H is square or invertible is needed to formulate this problem.

Least squares uses the observation model, but it does not explicitly incorporate prior knowledge about the signal. For example, we may know that x has a sparse wavelet representation. To include such information, introduce a penalty R and consider

$$\boxed{\min_{x\in\mathbb{R}^N} \left\{ \|y - \mathrm{H}x\|_2^2 + \lambda R(x) \right\}, \qquad \lambda > 0.} \tag{14.15}$$

This is *regularized least squares*. The penalty R is chosen so that signals consistent with the prior information have smaller penalty values. The parameter λ controls how strongly this preference is balanced against agreement with the observations. The formulation specifies what is being optimized; solving it depends on the properties of H and the particular penalty.

These formulations place signal recovery in the language of linear algebra and optimization. To connect them with the wavelet algorithms developed above, it remains to express the finite DWT in the same matrix notation.

14.16 The finite DWT as an orthogonal matrix

The DWT also has a finite-dimensional matrix representation. Fix the periodic boundary convention and a real input length $N = 2^I$. Filtering, downsampling, and arranging the output coefficients are linear operations. Since the full transform has N inputs and N outputs, there is an $N \times N$ matrix that represents it:

$$\boxed{\mathrm{DWT}\{x\} = c = \mathrm{W}x, \qquad \mathrm{W} \in \mathbb{R}^{N\times N}.} \tag{14.16}$$

The vector c contains the coarsest approximation and all detail coefficients in the order given in (14.8).

For real orthogonal wavelet filters and the corresponding periodic implementation, W is an orthogonal matrix:

$$\boxed{\mathrm{W}^\mathsf{T}\mathrm{W} = \mathrm{W}\mathrm{W}^\mathsf{T} = \mathrm{I}_N.} \tag{14.17}$$

This is the finite-dimensional counterpart of representing a signal in an orthonormal basis. In particular, the inverse transform is $x = \mathrm{W}^\mathsf{T}c$.

Orthogonality also gives the energy identity

$$\|\mathrm{W}x\|_2^2 = x^\mathsf{T}\mathrm{W}^\mathsf{T}\mathrm{W}x = x^\mathsf{T}x = \|x\|_2^2. \tag{14.18}$$

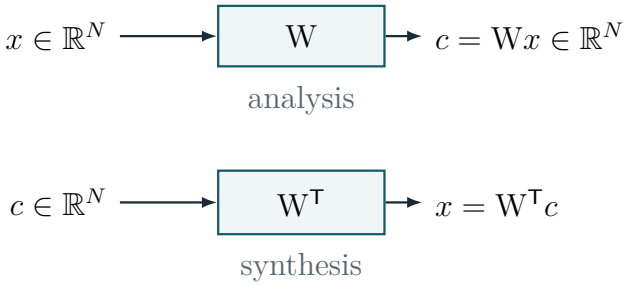


Figure 14.11: Analysis and synthesis for a real orthogonal periodic DWT. The analysis matrix is W, and the synthesis matrix is its transpose. Both maps are implemented by the filter-bank algorithm.

The matrix representation connects the filter-bank calculation to linear algebra. It does not prescribe a dense matrix implementation: the analysis bank evaluates the same map in $O(LN)$ operations. In a biorthogonal periodic DWT with perfect reconstruction, the transform matrix is invertible, but its inverse need not equal its transpose and it need not preserve the Euclidean norm.

We now have a common language for acquisition and representation: H maps a signal to its predicted measurements, while W maps it to wavelet coefficients. Sparsity describes structure in those coefficients that a recovery method can use. In the next lecture, we will study the least-squares and regularized formulations and the effect of particular choices of R.